ONNX·TensorRT Day 3

ONNX Runtime 으로 빠르게 — 제대로 재고, 하나씩 바꿔 비교하기

HYUNDAI AI Insight Campus · 온디바이스 AI · ONNX·TensorRT 3/5

2026. 11. 6 (금) 09:00–18:00 · 8교시 · Google Colab (T4 GPU 런타임)

bench() | p95 | threads | opt level | CUDA EP | profiling

강사 : 김정인 · 김구수

오늘의 학습 목표

4 단계 중 ' 실행하기 ' 를 빠르게 — ' 빠르다 ' 를 숫자로 말한다

1 **bench()**

warm-up · 반복 측정 · 평균 · p50 · p95 · FPS 로 추론 시간을 잰다

2 **스레드 수**

SessionOptions 로 스레드를 바꾸고 , 코어 수와의 관계를 설명한다

3 **최적화 레벨**

DISABLE → ALL 4 단계 차이를 노드 (FusedConv · FusedGemm) 와 시간으로 본다

4 **CPU → GPU**

CUDAExecutionProvider 로 실행하고 복사 · 첫 실행 · 결과 차이를 설명한다

5 **배치 크기**

지연 (ms) 과 처리량 (장 / 초) 을 구분하고 , 카메라 수를 계산한다

6 **profiling**

연산 종류별 시간 비율로 병목을 찾는다 — Day 4 의 출발점

시간표

8 교시 × 50 분 · 점심 12:00–13:00 · 매 교시 보기 → 따라하기 → 바꿔보기

교시	시간	주제	형태	애니메이션
1	09:00–09:50	Day 2 복습 퀴즈 · 오늘의 모델 · GPU 확인	퀴즈 + 실습	D3-1 오늘의 지도
2	10:00–10:50	시간을 재는 법 — warm-up · 평균 · p95 · FPS → bench()	강의 + 실습	D3-2 재는 법
3	11:00–11:50	스레드 수 — SessionOptions · 한 번에 하나만	강의 + 실습	D3-3 스레드 · 코어
4	13:00–13:50	그래프 최적화 레벨 — 노드 합치기 · 레이아웃	강의 + 실습	D3-4 최적화 레벨
5	14:00–14:50	CPU → GPU — CUDAExecutionProvider	강의 + 실습	D3-5 CPU → GPU
6	15:00–15:50	배치 크기 — 지연 vs 처리량	강의 + 실습	D3-6 지연 vs 처리량
7	16:00–16:50	profiling — 어디가 느린가	강의 + 실습	D3-7 profiling
8	17:00–17:50	미니 프로젝트 — 현장 카메라 4 대용 설정 정하기	개인 실습	D3-8 카메라 4 대

오늘의 자료 · 온라인 링크

예측 (애니메이션) → 확인 (노트북) → 점검 (워크시트)

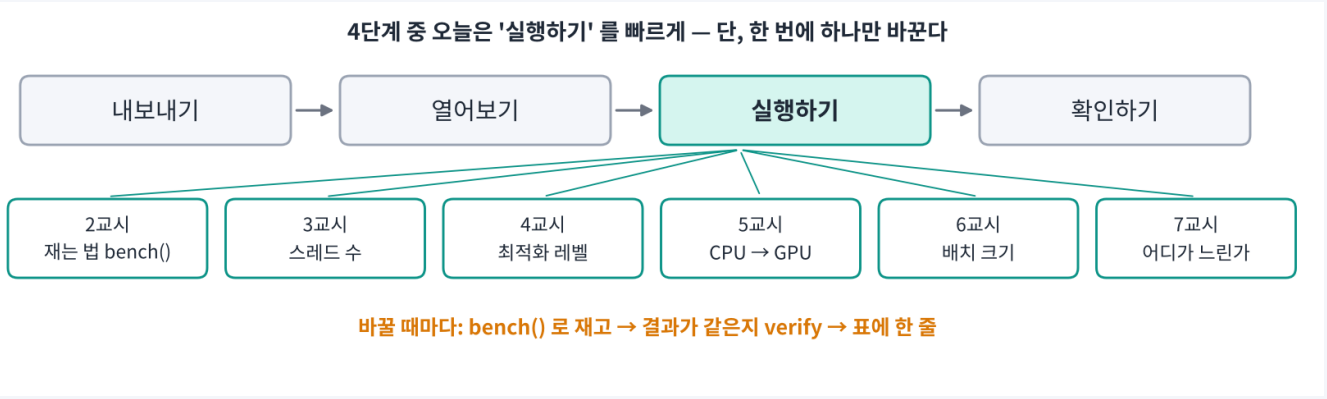
기본 주소 (Day 3)

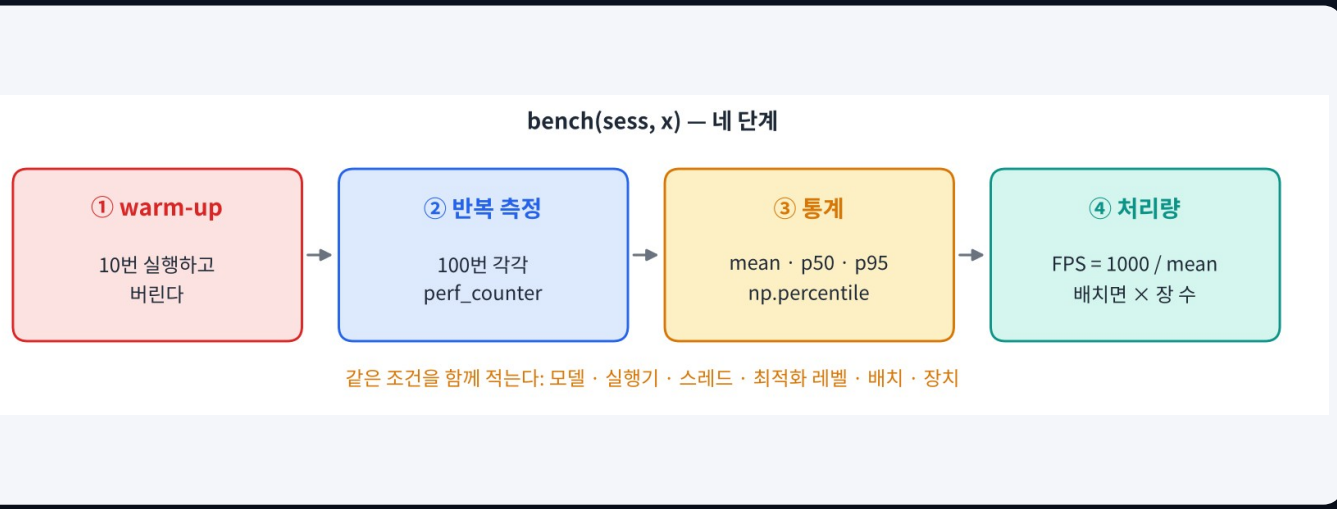
https://imguru-mooc.github.io/AI_CAMPUS/OT-day3/

animation.html	애니메이션 8 개 (D3-1 ~ D3-8) — #d3-1 처럼 바로가기
worksheet.html	개념 워크시트 — #t1 ~ #t8 교시 바로가기 (교시당 10 문항)
OT_Day3_lecture.ipynb	강의 노트북 — 1~8 교시 문제 + 미니 프로젝트 (Colab 에서 열기)
OT_Day3.pdf	오늘 슬라이드 PDF
Day 2 복습 퀴즈	1 교시 09:00 — Google Forms 20 문항 (강사가 링크 공유)
런타임 → T4 GPU	5 교시 GPU 실습용 — 노트북을 열자마자 바꾸기
onnxruntime-gpu	설치 셀이 CPU 판을 지우고 GPU 판을 설치 (약 1 분)

오늘의 큰 그림 — 손잡이 다섯 개를 하나씩

실험 원칙 : 한 번에 하나만 · 조건을 함께 적기 · 결과는 같게





오늘의 모델 · 실행 예 (Colab CPU)

mnv3s.onnx — 1–8 교시

MobileNetV3-Small · 노드 127 · opset 17

mnist_cnn_dyn.onnx — 2·4·7 교시

SimpleConvNet · 배치 자유

평균 1.9 ms — 2 교시

mnv3s 배치 1 · 약 530 FPS

ALL ≈ 2.5 배 — 4 교시

DISABLE 5.4 ms → ALL 2.2 ms

Conv ≈ 57% — 7 교시

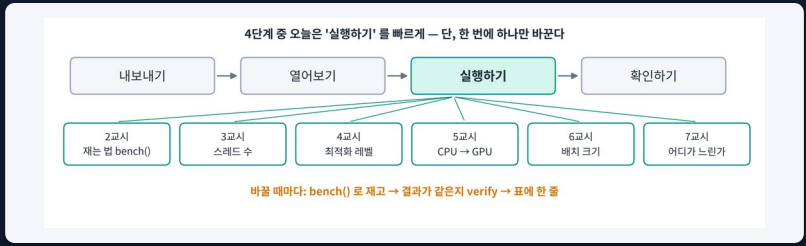
mnv3s 시간의 절반 이상

1 Day 2 복습 · 오늘의 모델 · GPU 확인

09:00–09:50

학습 목표

- Day 2 복습 퀴즈 20 문항 (15 분)
- mnv3s.onnx · opset 17 · 배치 자유
- providers = 희망 목록
- get_providers() 로 실제 확인



노트북 실습

1. 복습 퀴즈 (Google Forms)
2. 문제 1-1 오늘의 모델 두 개
3. 문제 1-2 GPU_OK 확인

▶ 애니메이션 D3-1 오늘의 지도 · 실험 원칙

핵심 개념

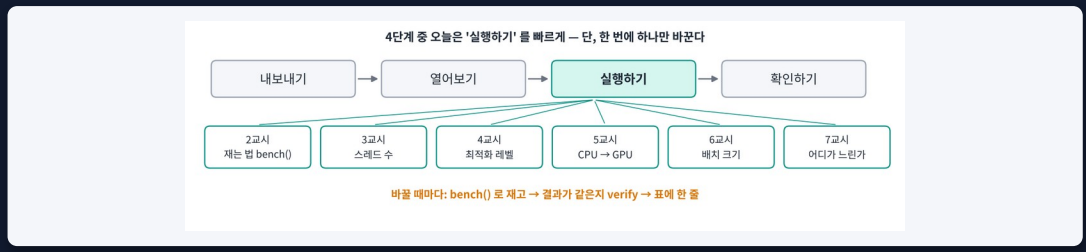
오늘의 모델

```
mnv3s = make_mnv3s()   # 구조만, BN 통계 seed 고정
torch.onnx.export(..., opset_version=17,
    dynamic_axes={"input": {0: "batch"}, ...})
```

GPU 확인

```
probe = ort.InferenceSession(path,
    providers=["CUDAExecutionProvider",
"CPUExecutionProvider"])
GPU_OK = probe.get_providers()[0] == "CUDAExecutionProvider"
```

오늘의 지도



체크포인트

CUDA 가 안 되면 오류 없이 CPU 로 넘어간다 — 세션을 만든 뒤 꼭 get_providers().

2 시간을 재는 법 → bench()

10:00–10:50

학습 목표

- perf_counter · 첫 실행은 느리다
- warm-up 10 번 버리고 100 번 재기
  - 평균 · p50 · p95 · FPS = 1000 ÷ 평균
  - 배치면 FPS × 장 수 = 장 / 초



노트북 실습

1. 문제 2-1 한 번씩 5 번
2. 문제 2-2 warm-up + 100 번
3. 문제 2-3 bench() 완성
4. 🔁 PyTorch vs ORT

▶ 애니메이션 D3-2 재는 법 — warm-up · p95

핵심 개념

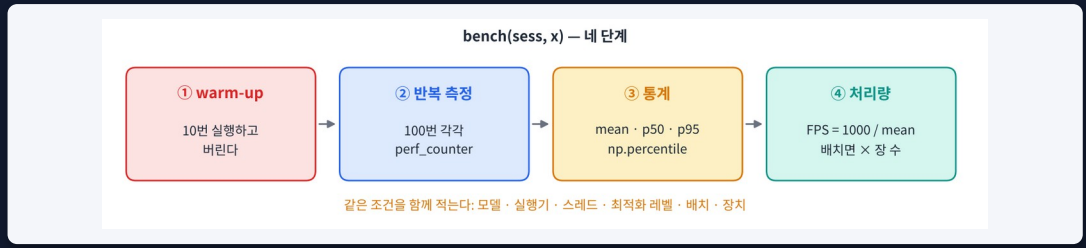
100 번

```
평균 1.90 ms | p50 1.89 | p95 2.08 | 526 FPS
# 실행 예 · Colab 장비마다 다름
```

bench

```
return {"mean": t.mean(),
        "p50": np.percentile(t, 50),
        "p95": np.percentile(t, 95),
        "fps": 1000 / t.mean() * x.shape[0]}
```

네 단계



체크포인트

실시간은 p95 로 판단 . PyTorch 6.3 ms vs ORT 1.9 ms (실행 예 약 3.4 배).

3 스레드 수 — SessionOptions

11:00–11:50

학습 목표

- so.intra_op_num_threads = n
  - 옵션 → 세션 순서 (나중 변경 무효)
  - 코어 수가 상한 · 넘으면 p95 부터
  - 기본값 0 = 코어 수에 맞춰 자동



노트북 실습

1. 문제 3-1 스레드 1 · 2 · 4 표
2. 문제 3-2 p95 기준 최선 (idxmin)
3. make_session() 도구

▶ 애니메이션 D3-3 스레드 · 코어

핵심 개념

옵션

```
so = ort.SessionOptions()
so.intra_op_num_threads = th
s = ort.InferenceSession("mnv3s.onnx", so, providers=CPU)
```

코어 2 개

```
threads  mean   p95
   1     1.87   1.99
   2     1.74   1.89
   4     3.96   6.10   ← 과밀
```

일꾼 · 작업대



체크포인트

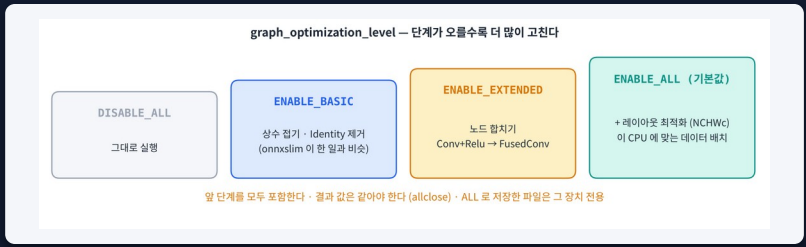
{"threads": th, **bench(...)} — 조건 + 결과를 한 줄로 . 오늘 표는 모두 이 방식 .

4 그래프 최적화 레벨

13:00–13:50

학습 목표

- DISABLE · BASIC · EXTENDED · ALL
- EXTENDED = 노드 합치기 (Fused*)
- ALL = 레이아웃 (NCHWc) · 기본값
- 레벨이 달라도 결과는 같다



노트북 실습

1. 문제 4-1 레벨별 노드 목록
2. 문제 4-2 mnv3s 레벨별 시간
3. 🔁 레벨별 allclose

▶ 애니메이션 D3-4 최적화 레벨 · 노드 합치기

핵심 개념

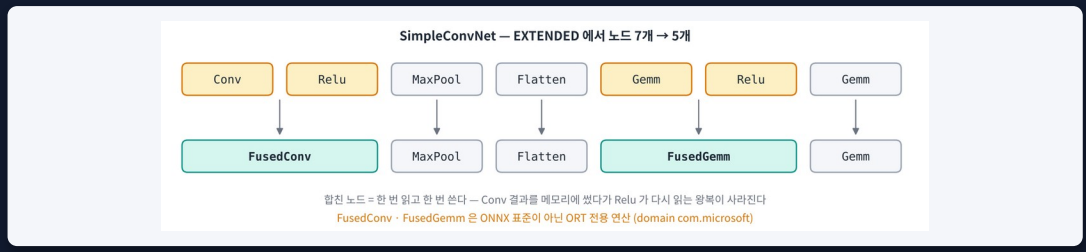
노드

```
DISABLE  7  Conv Relu MaxPool Flatten Gemm Relu Gemm
EXTENDED 5  FusedConv MaxPool Flatten FusedGemm Gemm
ALL      6  Conv(nchwc) ReorderOutput ...
```

시간

```
DISABLE 5.44 · BASIC 5.43 · EXTENDED 5.27
ALL 2.15 ms  →  약 2.5 배 ( 실행 예 )
```

합치기



체크포인트

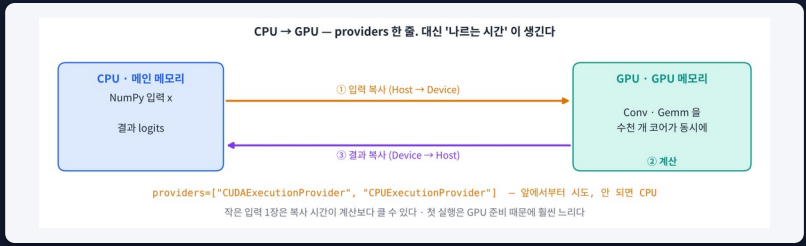
ALL 로 저장한 그래프는 그 장치 전용 — 배포는 원래 ONNX, 장치에서 세션을 만들 때 최적화 .

5 CPU → GPU — CUDAExecutionProvider

14:00–14:50

학습 목표

- onnxruntime-gpu 만 설치 (CPU 판 제거)
  - 복사 → 계산 → 복사
  - 첫 실행이 매우 느리다 → warm-up
  - 배치 1 vs 32 · atol 1e-3



노트북 실습

1. 문제 5-1 GPU 첫 5 번
2. 문제 5-2 CPU · GPU × 배치 1 · 32
3. 문제 5-3 GPU vs CPU 결과

▶ 애니메이션 D3-5 CPU → GPU

핵심 개념

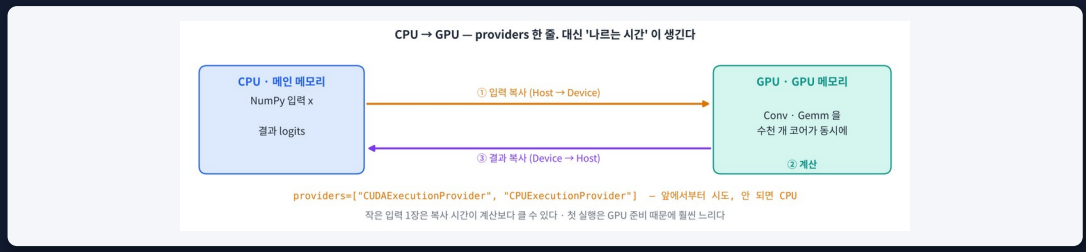
세션

```
gpu_sess = make_session("mnv3s.onnx",
    providers=["CUDAExecutionProvider",
               "CPUExecutionProvider"])
```

비교

```
CPU 배치 1   1.86 ms · 537 장 / 초
CPU 배치 32 64.6 ms · 495 장 / 초
GPU 줄은 Colab T4 에서 확인
```

복사 · 계산



체크포인트

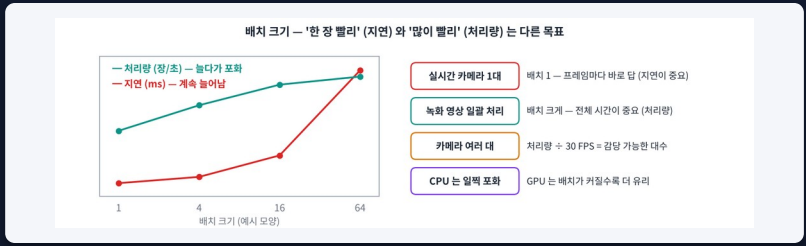
GPU 는 일이 많을수록 유리. 배치 1 만 재고 'GPU 가 별로' 라고 결론 내지 않기.

6 배치 크기 — 지연 vs 처리량

15:00–15:50

학습 목표

- 지연 = 한 번 run 시간 (ms)
- 처리량 = 배치 × 1000 ÷ 지연
- 실시간 → 배치 1 · p95
- 카메라 수 = 처리량 ÷ 30



노트북 실습

1. 문제 6-1 배치 1 · 4 · 16 · 64 표
2. 문제 6-2 그래프 · idxmax
3. 문제 6-3 카메라 몇 대 ?

▶ 애니메이션 D3-6 지연 vs 처리량

핵심 개념

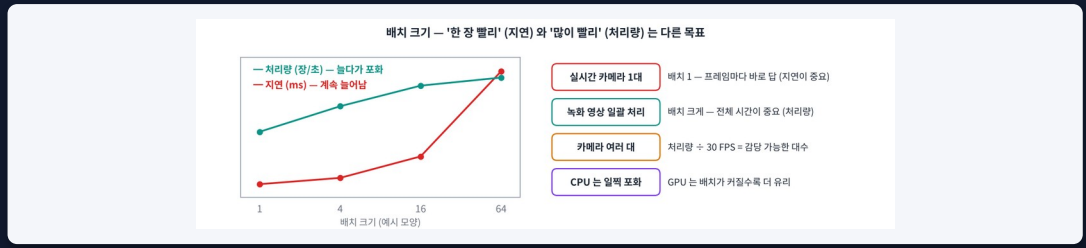
CPU 표

```
batch  latency_ms  imgs_per_s
   1       1.92         520
   4       6.78         590   ← 최대
  16      31.54         507
  64     138.06         464
```

카메라

```
cams_b1 = int(520 // 30)    # 17 대
cams_best = int(590 // 30)  # 19 대
```

두 곡선



체크포인트

처리량 최대 배치라도 지연이 예산 (30 FPS = 33 ms) 을 넘으면 실시간에는 못 쓴다 .

7 profiling — 어디가 느린가

16:00–16:50

학습 목표

- so.enable_profiling = True
- run → end_profiling() → JSON
- cat == "Node" · _kernel_time · dur µs
- op_name 별 합 → 시간 비율



노트북 실습

1. 문제 7-1 노드 기록 꺼내기
2. 문제 7-2 연산별 비율
3. 문제 7-3 profile_top() · 두 모델

▶ 애니메이션 D3-7 profiling — 병목 찾기

핵심 개념

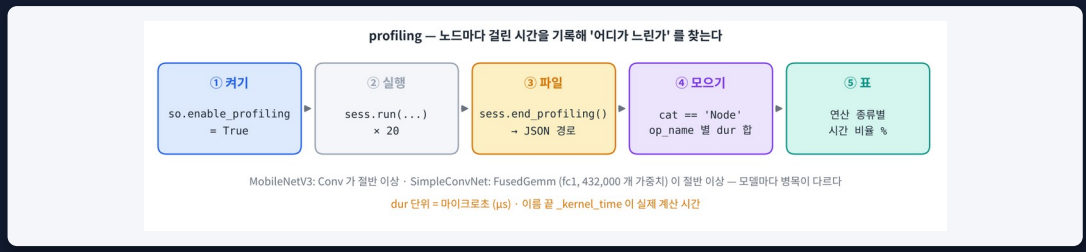
기록

```
prof_path = ps.end_profiling()
node_ev = [e for e in json.load(open(prof_path))
   if e.get("cat") == "Node"
   and e["name"].endswith("_kernel_time")]
```

비율

```
mnv3s   Conv 57% · Mul 14% · Gemm 10%
cnn     FusedGemm 61% · Conv 17% · MaxPool 8%
```

다섯 단계



체크포인트

재는 세션과 기록하는 세션은 따로 . 병목이 Conv → Day 4 TensorRT · Quantization.

8 미니 프로젝트 — 카메라 4 대용 설정

17:00–17:50

학습 목표

- 카메라 4 대 × 30 FPS = 120 장 / 초
- 스레드 {1, 2} × 레벨 {DISABLE, ALL}
- p95 최소 → 배치 1 · 4 · 8
- ot_baseline.csv → Day 4 로



노트북 실습

1. 과제 1 bench_config (20)
2. 과제 2 실험표 (20)
3. 과제 3 배치 · 처리량 (20)
4. 과제 4 profiling Top 3 (20)
5. 과제 5 · 6 CSV · 추천 (20)

▶ 애니메이션 D3-8 카메라 4 대 설정 정하기

핵심 개념

실험표

```
t1 DISABLE p95 6.62 · t1 ALL 2.23  ← 최선
t2 DISABLE p95 5.09 · t2 ALL 2.41
#  실행 예
```

배치

```
b1 473 · b4 606 · b8 497  장 / 초
120  이상 중 지연 최소 → 배치 1
```

흐름



체크포인트

채점 셀 자동 90 점 + 과제 6 서술 10 점 . 숫자가 아니라 방법 · 일관성을 채점 .

오늘의 핵심 정리

각 항목 옆 번호는 다시 볼 애니메이션

재는 법	warm-up 버리기 → n 번 → 평균 · p50 · p95 · FPS · 첫 실행은 성능이 아니다	D3-2
bench()	조건 + 결과 한 줄 · 배치면 FPS × 장 수 = 장 / 초	D3-2
스레드	SessionOptions → 세션 · 코어 수가 상한 · 넘으면 p95 부터 나빠짐	D3-3
최적화 레벨	BASIC 상수 접기 · EXTENDED Fused* · ALL 레이아웃 (기본값) · 결과는 같다	D3-4
GPU	CUDAExecutionProvider · get_providers() 확인 · 복사 · 첫 실행 · 큰 배치에 유리	D3-5
배치	지연 vs 처리량 · 실시간 = 배치 1 · 카메라 수 = 처리량 ÷ 30	D3-6
profiling	enable_profiling → end_profiling → op_name 별 dur · 병목은 모델마다	D3-7
실험 원칙	한 번에 하나만 · 조건을 열로 · 바꾼 뒤에도 allclose	D3-1

Day 4 로 가는 다리 — TensorRT 엔진

오늘 찾은 병목 (Conv) 을 GPU 전용으로 다시 조립한다

Day 3 (오늘)		Day 4 (다음 시간)
`bench() · ot_baseline.csv`	→	TensorRT 줄을 같은 열로 이어 붙이기
`CUDAExecutionProvider`	→	TensorrtExecutionProvider · trtexec 엔진
`ORT 최적화 레벨 (Fused*)`	→	TensorRT 의 레이어 합치기 · 커널 자동 선택
`FP32 · GPU vs CPU atol 1e-3`	→	FP16 — 더 빠르게 , 차이 기준 다시 정하기
`profiling: Conv ≈ 57%`	→	Conv 가 가장 많이 빨라지는지 확인
`mnv3s.onnx (opset 17)`	→	그대로 TensorRT 입력으로

노트북 · ot_baseline.csv 를 Drive 에 저장해 두세요 — Day 4 에서 이어 씁니다